OPONA Project Brief

Project Name

OPONA

Tagline

Know before you go.

Core Idea

OPONA is a real-time local availability app that helps South Africans find places that are actually open, active, and worth visiting right now.

The first MVP focuses on restaurants in Pretoria East, especially restaurants that are open past 20:00 and still serving food. The long-term brand must remain expandable beyond restaurants, so the app should not be designed as a food-only product.

Future categories could include pharmacies, cafés, hardware stores, vets, petrol stations, government offices, and other places where people need accurate availability information.

One-Sentence Pitch

OPONA helps users know whether a place is genuinely open, available, and worth the trip before they go.

Initial MVP Focus

The first version of OPONA should focus on:

Pretoria East restaurants that are open late, still serving food, and suitable for spontaneous visits.

The app should help answer questions like:

- Is this place open right now?
- Is the kitchen still serving?
- Is it open past 20:00?
- Are walk-ins available?
- Is it quiet, normal, busy, or packed?
- Is there backup power?
- Is the card machine working?
- When was this status last confirmed?

The app should not try to replace Google Maps. Instead, it should provide the real-time information Google often does not show clearly.

Product Positioning

OPONA is not a generic restaurant directory.

It is a live availability layer.

Google Maps helps users discover places. OPONA helps users decide whether it is actually worth going there right now.

Target User

Primary MVP users:

- Young adults and students in Pretoria East
- Couples or friend groups deciding where to eat at night
- People who want to avoid wasted trips
- Users who often search for restaurants after 20:00
- People who care about real-time status more than long reviews

Secondary future users:

- Restaurant owners
- Business managers
- Local service providers
- People looking for pharmacies, hardware stores, vets, cafés, or other open places

First Launch Area

Start with Pretoria East only.

Suggested areas:

- Menlyn
- Hazelwood
- Brooklyn
- Lynnwood
- Faerie Glen
- Moreleta Park
- Garsfontein
- Waterkloof
- The Willows
- Equestria
- Silver Lakes

- Olympus

The app should support area-based filtering from the start.

Core User Problem

A user wants to go out for food at night.

Google says a restaurant is open until 22:00.

The user drives there at 21:00.

The restaurant is technically open, but the kitchen closed at 20:30, or it is fully booked, too busy, affected by load shedding, or not accepting walk-ins.

OPONA exists to prevent this wasted trip.

MVP Feature List

1. Restaurant Listings

Show a list of Pretoria East restaurants with key information.

Each restaurant card should show:

- Restaurant name
- Area
- Cuisine type
- Distance, if location is available
- Opening status
- Kitchen status
- Closing time
- Last confirmed time
- Confidence level
- Quick status chips
- Primary action button

Example restaurant card:

Culture Club
Hazelwood · Mediterranean · 2.4 km away

Serving now
Kitchen confirmed 14 min ago

Chips:

- Walk-ins likely
- Busy
- Generator
- Card working
- Kitchen closes 21:30

CTA: Get directions

2. Filters

The app must support clear, fast filters.

Important MVP filters:

- Area
- Open now
- Open past 20:00
- Open past 21:00
- Kitchen serving
- Walk-ins available
- Quiet
- Normal
- Busy
- Has backup power
- Card machine working
- Outdoor seating
- Family friendly
- Cuisine type
- Distance
- Last confirmed recently

Area filters should include Pretoria East suburbs.

3. Restaurant Detail Page

Each restaurant should have a detail page or bottom sheet.

Show:

- Name
- Area
- Address
- Cuisine
- Current status
- Kitchen status

- Opening hours
- Kitchen hours, if known
- Last confirmed time
- Confidence level
- Phone number
- Directions button
- Website or menu link, if available
- Status history summary
- User confirmation section

4. Live Status System

OPONA should not only show static opening hours.

Each place should have a live status object.

Possible status fields:

- isOpen
- isKitchenServing
- acceptsWalkIns
- busyLevel
- hasBackupPower
- cardMachineWorking
- lastConfirmedAt
- confidenceScore
- statusSource

Busy level options:

- quiet
- normal
- busy
- packed
- unknown

Confidence options:

- high
- medium
- low
- unknown

Status wording examples:

- Confirmed recently
- Likely open
- Kitchen not confirmed yet

- Reported closed
- Status needs confirmation

5. Community Confirmations

Users should be able to confirm status with one tap.

Avoid long reviews in the MVP.

Use structured confirmation questions.

Examples:

Question: Is the kitchen still serving?

Options:

- Yes
- No
- Not sure
- I did not go

Question: How busy is it?

Options:

- Quiet
- Normal
- Busy
- Packed

Question: Could you get a table?

Options:

- Yes
- No, fully booked
- Wait too long
- Did not try

Question: Is load shedding affecting them?

Options:

- Operating normally
- Limited service
- Closed

- Not sure

Question: Is the card machine working?

Options:

- Yes
- No
- Cash only
- Not sure

6. Smart Notifications

Notifications should be used carefully and only when relevant.

The app should not spam users.

Good notification triggers:

- User clicked directions to a restaurant
- User clicked call
- User searched for a restaurant open now
- User was near a restaurant after interacting with it
- User saved a restaurant
- User visited during late hours
- User visited on a public holiday

Notification examples:

After navigation: Were they still serving at Culture Club? Options:

- Yes
- No
- Did not go

After call: Did they answer? Options:

- Yes, open
- No answer
- Number invalid
- Not sure

After visit: How busy was it? Options:

- Quiet
- Normal
- Busy
- Packed

Public holiday: Do you know if this place is open today? Options:

- Open
- Closed
- Not sure

Rules:

- Ask at most once per day by default.
- Only ask after high-intent user actions.
- Always include a skip option.
- Notifications must be opt-in.
- Location-based prompts must require clear user permission.

7. Confidence Score

Each live status should have a confidence score.

The confidence score should be affected by:

Positive factors:

- Recent user confirmation
- Multiple matching confirmations
- User was physically near the place
- User clicked directions before confirming
- Business owner confirmation
- Status matches official hours
- Reporter has a history of reliable confirmations

Negative factors:

- Old confirmation
- Conflicting reports
- User was far away
- User submits too many reports too quickly
- Status contradicts many recent confirmations
- No recent data available

Example display:

Kitchen serving
Confirmed 14 min ago
High confidence

or:

Kitchen status unknown
Last confirmed 6 hours ago
Low confidence

8. Business Claiming

Not required for the first coded prototype, but the product should be designed with future business profiles in mind.

Future claimed businesses should be able to update:

- Current status
- Special hours
- Kitchen hours
- Public holiday hours
- Temporary closures
- Backup power status
- Card machine status
- Walk-in availability
- Busy level
- Special offers
- Event notices

Business dashboard features can come later.

9. Public Holiday Mode

Future feature, but design the data model with this in mind.

Public holiday mode should help users find places actually open on public holidays.

Possible SEO pages later:

- Restaurants open on public holidays in Pretoria East
- Restaurants open on Christmas Day in Pretoria
- Restaurants open on Easter Monday in Pretoria East
- Places open now in Pretoria East

Brand Identity

Brand Name

OPONA

The name hints at “open” but remains broad enough to expand beyond restaurants.

Brand Personality

OPONA should feel:

- Calm
- Useful
- Premium
- Local
- Trustworthy
- Slightly warm
- Night-friendly
- Practical, not gimmicky

It should not feel like:

- A food delivery app
- A loud discount app
- A generic restaurant directory
- A review-heavy social platform
- A government-style utility

Visual Metaphor

A warm light in a dark street.

The app should feel like it helps users find places where the lights are still on.

Colour Scheme

Use a dark, warm, premium palette.

Primary palette:

Background:

17151C

Surface:

24212B

Primary amber:

FFB84D

Accent coral:

F26B4F

Success green:

59D68C

Warning yellow:

FFD166

Closed or error red:

EF5D60

Primary text:

FFF8ED

Muted text:

B9AFA3

Border:

393441

The amber should represent warmth, restaurant lighting, and places being open.

The green should be used carefully for confirmed open states.

The red should only be used for closed or problem states.

Typography

Use this font pairing:

Headings and brand: Gloock

Body and UI: Instrument Sans

Usage rules:

Use Gloock for:

- Logo wordmark
- Hero headings
- Large page headings
- Marketing sections
- Empty states

Use Instrument Sans for:

- Body text
- Buttons
- Filter chips
- Cards
- Forms
- Navigation
- Status labels
- Restaurant names in dense lists
- Confirmation prompts

Rule:

Gloock sells the feeling. Instrument Sans runs the product.

UI Design Direction

The design should be:

- Mobile-first
- Dark by default
- Card-based
- Status-first
- Filter-heavy
- Clean and spacious
- Slightly premium
- Warm and night-friendly

Core UI ideas:

- Rounded cards
- Sticky filter chips

- Bottom navigation on mobile
- Map/list toggle
- Large tap targets
- Clear status badges
- Confidence indicators
- One-tap confirmation prompts
- Warm glowing accents
- Minimal clutter

Suggested App Structure

Main User Routes

- /
- /restaurants
- /restaurants/:id
- /areas/:areaSlug
- /confirm/:restaurantId
- /saved
- /profile
- /settings

Future Business Routes

- /business
- /business/claim
- /business/dashboard
- /business/restaurants/:id

SEO Routes for Later

- /restaurants/open-now/pretoria-east
- /restaurants/open-after-20/pretoria-east
- /restaurants/open-after-21/pretoria-east
- /areas/menlyn/restaurants-open-now
- /areas/hazelwood/restaurants-open-now
- /areas/brooklyn/restaurants-open-now

Recommended Tech Stack

Frontend:

- React
- TypeScript
- Vite
- React Router
- CSS Modules or Tailwind CSS
- PWA support

- Responsive design for mobile and desktop

Desktop:

- Electron wrapper around the React app
- The same React UI should run as both web PWA and desktop app

Maps:

- Use Leaflet with OpenStreetMap for MVP
- Avoid paid map APIs until the product is validated

Backend options:

Option A for fast MVP:

- Mock JSON data first
- Local storage for saved restaurants
- Simulated confirmation data

Option B for proper backend:

- Node.js and Express
- PostgreSQL
- Prisma ORM

Option C if using a backend-as-a-service:

- Supabase
- PostgreSQL
- Supabase Auth
- Supabase Edge Functions later if needed

Recommended MVP approach:

Start with React, TypeScript, Vite, mock data, and local state. Add backend only after the main UX is working.

Suggested Folder Structure

src/ assets/ components/ AppShell/ BottomNav/ FilterChips/ RestaurantCard/ RestaurantStatusBadge/ ConfidenceBadge/ AreaSelector/ ConfirmationPrompt/ SearchBar/ MapView/ data/ mockRestaurants.ts areas.ts filters.ts hooks/ useFilteredRestaurants.ts useUserLocation.ts useConfirmationPrompt.ts layouts/ MobileLayout.tsx DesktopLayout.tsx pages/ HomePage.tsx RestaurantsPage.tsx RestaurantDetailPage.tsx SavedPage.tsx SettingsPage.tsx styles/ variables.css global.css types/ restaurant.ts status.ts filters.ts utils/ time.ts status.ts confidence.ts distance.ts App.tsx main.tsx

Suggested TypeScript Types

Restaurant:

id: string name: string slug: string area: string address: string latitude: number longitude: number cuisine: string[] phone?: string website?: string menuUrl?: string openingHours: OpeningHours[] kitchenHours?: OpeningHours[] features: RestaurantFeature[] currentStatus: LiveStatus

OpeningHours:

day: string openTime: string closeTime: string isClosed: boolean

LiveStatus:

isOpen: boolean isKitchenServing: boolean | null acceptsWalkIns: boolean | null busyLevel: "quiet" | "normal" | "busy" | "packed" | "unknown" hasBackupPower: boolean | null cardMachineWorking: boolean | null lastConfirmedAt: string | null confidenceScore: number source: "official" | "community" | "business" | "estimated" | "unknown"

RestaurantFeature:

"outdoor_seating" "family_friendly" "wheelchair_accessible" "backup_power" "card_payments" "late_night" "walk_ins" "takeaway" "bar" "halaal_friendly" "vegan_options" "pet_friendly"

Confirmation:

id: string restaurantId: string userId?: string type: "open_status" | "kitchen_status" | "busy_level" | "walk_ins" | "card_machine" | "backup_power" value: string createdAt: string locationVerified: boolean source: "user" | "business"

Filtering Logic

The restaurant list should be filterable by:

- selected area
- current open status
- open after selected time
- kitchen serving
- cuisine
- busy level
- features
- confidence level
- distance

Example filter behaviour:

If the user selects “Open past 20:00”, show restaurants where the closing time is later than 20:00.

If the user selects “Kitchen serving”, show restaurants where isKitchenServing is true or recently confirmed.

If status is unknown, do not hide the restaurant by default unless the user selects “confirmed only”.

Status Display Rules

Do not overpromise.

If the app is unsure, say so clearly.

Good wording:

- Serving now
- Likely serving
- Kitchen not confirmed
- Confirmed open
- Reported closed
- Open, but status is old
- Closes soon
- Needs confirmation

Avoid:

- Definitely open
- Guaranteed open
- 100 percent accurate
- Verified if it is not actually verified

Privacy and Safety Requirements

Location access must be optional.

The app must ask for permission before using location.

Do not track users constantly.

Do not store precise location history unless it is necessary and clearly explained.

Confirmation prompts should be based on consent.

Users should be able to disable notifications.

For MVP, avoid accounts unless needed.

Use anonymous or local confirmations first if possible.

MVP Screens

1. Home Screen

Purpose: Introduce OPONA and guide users into the main restaurant search.

Content:

- OPONA logo
- Tagline: Know before you go.
- Search input
- Main CTA: Find places open now
- Secondary CTA: Browse Pretoria East
- Quick filters:
- Open now
- Open after 20:00
- Serving now
- Quiet spots
- Backup power

2. Restaurants Screen

Purpose: Main browsing and filtering screen.

Layout:

Mobile:

- Search at top
- Horizontal filter chips
- Restaurant cards
- Map/list toggle
- Bottom navigation

Desktop:

- Left filter sidebar
- Main restaurant list
- Right detail or map panel

3. Restaurant Detail Screen

Purpose: Show deeper status and actions.

Include:

- Restaurant name
- Area
- Status summary
- Kitchen status
- Busy level
- Hours
- Features
- Last confirmed
- Confidence level
- Directions button
- Call button
- Confirm status button

4. Confirmation Prompt

Purpose: Collect real-time status data.

Design:

- One question at a time
- Large buttons
- No typing required
- Always include “Not sure” or “Did not go”

5. Saved Screen

Purpose: Let users save favourite places.

Saved places can later be used for helpful prompts and alerts.

6. Settings Screen

Purpose: Allow users to manage:

- Location permission
- Notifications
- Theme preference
- Saved areas
- Privacy settings

First Build Tasks for Codex

Build the MVP in stages.

Stage 1: Project setup

Create a React + TypeScript + Vite app.

Set up:

- React Router
- global CSS variables
- font imports
- base layout
- dark theme
- mock restaurant data

Stage 2: Core UI

Build:

- App shell
- Home page
- Restaurants page
- Restaurant card
- Filter chips
- Area selector
- Status badges
- Confidence badge

Stage 3: Filtering

Build filtering logic for:

- area
- open now
- open after 20:00
- kitchen serving
- busy level
- backup power
- card working
- cuisine

Stage 4: Detail screen

Build restaurant detail page with:

- route-based restaurant ID
- status summary
- full opening hours
- feature list
- confirmation button

- directions and call links

Stage 5: Confirmation system

Build local mock confirmation flow.

When a user confirms a status:

- update local state
- update lastConfirmedAt
- update confidenceScore
- show thank-you message

Stage 6: PWA setup

Add:

- manifest
- app icon placeholders
- service worker
- installability support
- mobile-safe responsive layout

Stage 7: Electron wrapper

Wrap the same React app in Electron.

Desktop app should support:

- larger layout
- left filter sidebar
- restaurant list
- detail panel

Important MVP Constraints

Do not build everything at once.

The first working prototype only needs:

- Pretoria East restaurants
- mock data
- restaurant list
- filters
- status cards
- restaurant detail screen
- simple confirmation flow

- responsive mobile and desktop layout

Do not build yet:

- user accounts
- real push notifications
- business dashboard
- payment system
- admin panel
- full database
- AI features
- nationwide search

Those can come later.

Sample Mock Restaurants

Create mock restaurants for Pretoria East areas.

Include realistic but placeholder data for:

- Hazelwood
- Menlyn
- Brooklyn
- Lynnwood
- Faerie Glen
- Moreleta Park
- Garsfontein
- Waterkloof

Each restaurant should have different combinations of:

- open now
- open after 20:00
- kitchen serving
- busy level
- backup power
- card machine working
- last confirmed time
- confidence score

Use clearly fake or placeholder data until real data is collected.

UI Copy Examples

Main heading:

Know before you go.

Subheading:

Find places nearby that are open, active, and worth the trip.

CTA:

Find open places

Filter labels:

- Open now
- Serving now
- Open after 20:00
- Walk-ins
- Quiet
- Busy
- Backup power
- Card working

Status labels:

- Serving now
- Kitchen not confirmed
- Confirmed open
- Likely open
- Reported closed
- Closes soon
- Last confirmed 14 min ago

Confirmation prompts:

Were they still serving? Was it busy? Could you get a table? Was the card machine working? Were they operating normally?

Thank-you message:

Thanks. You helped someone avoid a wasted trip.

Design Goal

The app should feel like a premium, warm, night-friendly local discovery tool.

It should be clean enough for quick decisions and beautiful enough to feel brandable.

The design should support the idea of a warm light in a dark street.

Final Product Vision

OPONA should eventually become the trusted South African app for live availability.

The restaurant MVP is only the first wedge.

Long term, OPONA can expand into any category where users ask:

Is it open? Is it available? Is it worth going? Can I trust the status?

The brand should always remain broader than restaurants.